

Módulo V: Taller Integrativo

Clase 3 — Entorno y primeros pasos en Streamlit

2026-07-01

Contacto

-  sebaegana@gmail.com
 -  <https://segana.netlify.app>
 -  <https://www.linkedin.com/in/sebastian-egana-santibanez/>
 -  <https://github.com/sebaegana>
-

Módulo V

Clase 3: Entorno y primeros pasos en Streamlit

VS Code · Ollama · Qwen · vibe coding

Objetivo de la sesión

- Instalar y configurar el entorno de trabajo: VS Code + Ollama + Continue.dev
 - Entender qué es vibe coding y cómo usarlo para aprender más rápido
 - Construir la primera app Streamlit con los KPIs del proyecto
 - Tener la app corriendo en `localhost` al final de la clase
-

El stack de esta clase

VS Code	← editor de código
+	
Ollama	← servidor de LLM local (sin internet, sin cuenta)
+	
Qwen2.5-Coder	← modelo especializado en código
+	
Continue.dev	← extensión que conecta el LLM con VS Code
+	
Streamlit	← framework para construir la app

Paso 1 — Instalar VS Code

1. Ir a code.visualstudio.com
 2. Descargar e instalar para Windows
 3. Extensiones recomendadas:
 - **Python** (Microsoft)
 - **Continue** (Continue.dev)
-

Paso 2 — Instalar Ollama

1. Ir a ollama.com y descargar el instalador para Windows
2. Instalar — Ollama corre como servicio en segundo plano
3. Abrir una terminal y descargar el modelo:

```
ollama pull qwen2.5-coder:7b
```

El modelo pesa ~4GB, se descarga una sola vez.

¿Computador con menos de 8GB RAM? Usar `qwen2.5-coder:1.5b` — pesa ~1GB y sigue siendo útil para código Python.

Paso 3 — Configurar Continue.dev en VS Code

1. En VS Code → Extensions → buscar **Continue**
 2. Instalar y abrir el panel de Continue (ícono en la barra lateral)
 3. Seleccionar modelo: **Ollama** → **qwen2.5-coder:7b**
 4. Probar: escribir en el chat "escribe una función que sume dos números" → el modelo responde con código
-

¿Qué es vibe coding?

Programar describiendo en lenguaje natural lo que quieres y dejando que el LLM genere el código.

Sin vibe coding

Buscar documentación → leer ejemplos → escribir código → depurar errores → repetir

Con vibe coding

Describir lo que quieres → el LLM genera el código → entender qué hace → ajustar → avanzar

El objetivo no es no aprender — es aprender más rápido viendo código funcional que puedes modificar.

Cómo describir bien lo que quieres

Un buen prompt para vibe coding incluye:

1. **Qué hace la app** — contexto general
2. **Qué quieres en este paso** — acción específica
3. **Con qué datos** — estructura del CSV o DataFrame
4. **Qué librerías usar** — Streamlit, Pandas, Plotly

Ejemplo:

“Tengo un CSV con columnas: indicador, valor, meta, perspectiva, periodo. Crea una app Streamlit que muestre una tarjeta st.metric por cada indicador del periodo seleccionado en un st.selectbox del sidebar.”

Paso 4 — Instalar Streamlit

En la terminal de VS Code:

```
pip install streamlit pandas plotly
```

Verificar que funciona:

```
streamlit hello
```

Se abre una app de demostración en el browser — eso confirma que la instalación está correcta.

Primera app — estructura mínima

```
# app.py
import streamlit as st
import pandas as pd

st.set_page_config(page_title="Dashboard KPI", layout="wide")

# Carga de datos
df = pd.read_csv("datos_kpis.csv")

# Título
st.title("Dashboard de Control de Gestión")

# Tabla de datos
st.dataframe(df, use_container_width=True)
```

Correr con:

```
streamlit run app.py
```

Agregar métricas con st.metric

```
import streamlit as st
import pandas as pd

df = pd.read_csv("datos_kpis.csv")
df_ult = df[df["periodo"] == df["periodo"].max()]

st.title("KPIs del período")

cols = st.columns(len(df_ult))

for col, (_, row) in zip(cols, df_ult.iterrows()):
    delta = row["valor"] - row["meta"]
    col.metric(
        label=row["indicador"],
        value=f"{row['valor']} {row['unidad']}",
        delta=f"{delta:+.1f} vs meta",
    )
```

Trabajo de la sesión

1. Instalar VS Code + Ollama + Qwen2.5-Coder + Continue.dev
2. Instalar Streamlit y verificar con `streamlit hello`

3. Crear `app.py` con la app mínima (título + tabla)
 4. Agregar las tarjetas `st.metric` con los KPIs del proyecto
 5. Usar Continue.dev para pedirle al LLM que agregue algo nuevo
-

Próxima clase

Clase 4 — Dashboard completo con vibe coding

- Sidebar con filtros interactivos
- Gráficos de tendencia y cumplimiento con Plotly
- Semáforo aplicado a cada indicador
- Cada grupo amplía su app con vibe coding